

Taller Práctico: Asincronía y Callbacks en JavaScript

Objetivo

Implementar y analizar soluciones utilizando callbacks y tareas asíncronas en JavaScript para comprender cómo funciona el comportamiento no bloqueante del lenguaje.

Parte 1. Callbacks Básicos

Un callback es una función que se envía como argumento a otra función para ser ejecutada posteriormente.

Analiza el siguiente código y completa las secciones marcadas con TODO.

```
function realizarOperacion(num1, num2, operacionCallback) {
    console.log(`Operación con: ${num1} y ${num2}`);
    operacionCallback(num1, num2);
}

function sumar(a, b) {
    console.log(`Resultado Suma: ${a + b}`);
}

// TODO
function restar(a, b) {

}

// TODO
function multiplicar(a, b) {

}

realizarOperacion(10, 5, sumar);

// TODO
// Implementar las tres operaciones.
```

Actividades

1. Implementa el callback `restar`.
2. Implementa el callback `multiplicar`.
3. Ejecuta las tres operaciones.
4. Captura la salida obtenida en consola.

```
> 'Operación con: 10 y 5'  
> 'Resultado Suma: 15'  
> 'Operación con: 10 y 5'  
> [ 'Resultado Resta:', 5 ]  
> 'Operación con: 10 y 5'  
> [ 'Resultado Multiplicar:', 50 ]
```

Preguntas de reflexión

1. ¿Qué ventaja tiene enviar una función como parámetro?

Permite que el código sea más reutilizable lo cual ayuda a evitar repetir el mismo código

2. ¿Qué ocurre si enviamos una función diferente como callback?
El comportamiento de la función cambia según la instrucción de ese callback
3. ¿La función principal necesita conocer qué operación matemática se ejecutará?

Si porque entonces la función principal no sabrá qué operación realizará

Parte 2. Comprendiendo la Asincronía

Observa cuidadosamente el siguiente programa.

```
function tareaNoBloqueante(callback) {  
  console.log("Iniciando tarea no bloqueante...");  
  
  setTimeout(function() {  
    console.log("Tarea completada.");  
    callback();  
  }, 2000);  
}  
  
console.log("Inicio del programa.");  
sdf  
tareaNoBloqueante(function() {  
  console.log("Continuando después de la tarea.");  
});  
  
console.log("Fin del programa.");
```

Actividad

Antes de ejecutar el programa:

1. Escribe el orden que crees que aparecerá en la consola.

“iniciando tarea no bloqueante”

“tarea completada”

“inicio del programa”

“continuando despues de la tarea”

“fin del programa”

2. Ejecuta el código.
3. Registra el resultado real.

“inicio del programa”

“iniciando tarea no bloqueante”

“fin del programa”

“tarea completada”

“continuando despues de la tarea”

```
> 'Inicio del programa.'  
> 'Iniciando tarea no bloqueante...'  
> 'Fin del programa.'  
> 'Tarea completada.'  
> 'Continuando después de la tarea.'
```

Análisis

Responde:

1. ¿Coincidió tu predicción con el resultado?

No

2. ¿Por qué el mensaje "Fin del programa" aparece antes que "Tarea completada"?
3. Porque `setTimeout()` es una función asincrónica y no bloquea la ejecución del programa.
4. ¿Qué papel cumple `setTimeout()` en este comportamiento?

Programa la ejecución de una función después de cierto tiempo sin detener el resto del programa

Parte 3. Simulación de Procesos Reales

Debes simular tres tareas comunes de una aplicación moderna.

Código Base

```
function solicitarJSON(callback) {  
  
}  
  
function reproducirAudio(callback) {  
  
}  
  
function leerSensor(callback) {  
  
}  
  
console.log("--- Iniciando procesos asíncronos ---");  
  
solicitarJSON(() => console.log("Archivo JSON recibido."));  
reproducirAudio(() => console.log("Audio finalizado."));  
leerSensor(() => console.log("Datos del sensor listos."));
```

Requisitos

Implementa cada función utilizando `setTimeout()`.

Función	Tiempo
solicitarJSON	3000 ms
reproducirAudio	1000 ms
leerSensor	500 ms

Actividades

1. Implementa las tres funciones.
2. Ejecuta el programa.
3. Registra el orden real de finalización.

Resultado

```
> '--- Iniciando procesos asíncronos ---'  
> 'Tarea completada.'  
> Datos del sensor listos.  
> 'Tarea completada.'  
> Audio finalizado.  
> 'Tarea completada.'  
> Archivo JSON recibido.
```

Orden	Tarea
1	leerSensor
2	reproducirAudio
3	solicitarJSON

Preguntas

1. ¿Cuál terminó primero?

leerSensor

2. ¿Cuál terminó último?

solicitarJSON

3. ¿Por qué el orden de finalización es diferente al orden de ejecución?

Porque el `setTimeout()` le dice a la función cuánto debe tardar en ejecutarse sin importar si se ejecuta primero o no

Parte 4. Diseñando tu Propia Operación Asíncrona

Reto

Crea una función llamada:

`simularDescarga(nombreArchivo, callback)`

La función deberá:

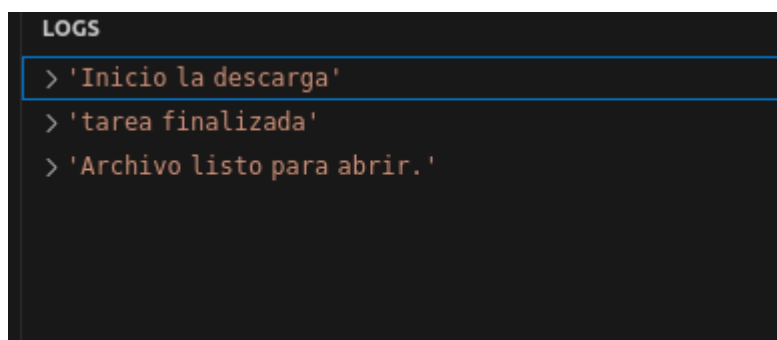
1. Mostrar un mensaje indicando que inició la descarga.
2. Esperar 4 segundos utilizando `setTimeout()`.
3. Mostrar un mensaje indicando que la descarga terminó.
4. Ejecutar el callback recibido.

Ejemplo esperado

```
simularDescarga("manual.pdf", () => {  
  console.log("Archivo listo para abrir.");  
});
```

Evidencia

Captura la salida de consola y explica qué sucede en cada paso.



Primero se imprime el mensaje “inicio de descarga” después la ejecución de la función tarda 4000ms (4seg) e imprime “tarea finalizada” y “archivo listo para abrir”

Parte 5. Análisis del Callback Hell

Observa la siguiente estructura:

```
obtenerUsuario(() => {  
  obtenerPedidos(() => {  
    obtenerFactura(() => {  
      enviarCorreo(() => {
```

```
        console.log("Proceso finalizado");
    });
});
});
});
```

Actividades

1. Describe el problema visual que presenta el código.

El código presenta funciones dentro de otras funciones (anidación) lo que genera que el código se desplace a la derecha

2. Explica qué es el Callback Hell.

Es una situación en la que varias operaciones asincrónicas dependen unas de otras, cada paso depende del anterior

3. ¿Por qué dificulta el mantenimiento de aplicaciones grandes?
esto puede volver el código confuso y difícil de mantener

4. Investiga y menciona dos alternativas modernas para evitar este problema.

1. con promesas usando `.then()` y `.catch()`

2. con `async` y `await`

Entregables

El estudiante deberá entregar:

- Código fuente de todos los ejercicios.
- Capturas de consola de las ejecuciones.
- Respuestas a las preguntas de análisis.
- Explicación del ejercicio de Callback Hell.